Runtime Governability: A Practitioner Discussion Paper

On the question raised by Szilvia Földesné Ambrus

"Evidence may persist. Authority may still exist. But when context changes, the admissibility of continued execution should not necessarily persist by inheritance."

Purpose of this paper: This is a focused discussion document, not a product brochure, not a capability inventory, and not a release history. It exists to examine one question honestly, against the actual OMG codebase, and to propose the minimum evolution needed to answer it more completely. Every claim about current OMG behavior in this paper is grounded in code that was read and verified; nothing here is aspirational unless explicitly marked as a proposal.

Section 1 — Understanding the Challenge

Three related but distinct ideas

Traditional Governance asks: *was this asset properly assessed, approved, and documented before it went into operation?* It is a point-in-time gate — a decision, a signature, a certificate. Once granted, it is treated as durable. This is the model most governance platforms, including most of OMG's original capability set, were built around: register the asset, assess its risk, assign owners, collect evidence, record a decision.

Governance Continuity asks a second question on top of the first: *does the original decision stay current over time?* This is where concepts like review cycles, reauthorization dates, and reassessment triggers come in. Continuity governance accepts that a governance decision has a shelf life and tries to track when that shelf life expires — but it still treats "has this expired yet?" as the whole question, generally checked on a calendar cadence (e.g., quarterly review) rather than continuously.

Runtime Governability asks a third, harder question: *right now, given everything that has changed since the decision was made — the model, the data, the tooling, the owner, the risk profile, the operating context — is it still legitimate for this asset to keep running, and would any intervention (escalation, reassessment, suspension, kill-switch) be admissible if someone chose to act?* This is not a calendar question. It is a standing, continuously-re-derivable question that should, in principle, be answerable at any moment, not just at the next scheduled review.

Szilvia's challenge, restated plainly

The core observation is that governance artifacts — evidence, authority, policy — are typically treated as **inherited**: once evidence is on file, it "counts" as evidence forever unless something explicitly expires it. Once an owner is assigned, they are "the owner" indefinitely. Once a decision is recorded, it stays valid indefinitely unless someone remembers to revisit it.

But governance artifacts are not automatically transitive across a changed context. Three concrete illustrations:

- **Evidence that still exists may no longer be sufficient.** A validation report written against version 1 of a model does not, on its own, tell you anything trustworthy about version 3 of that model — even though the file is still sitting in the evidence registry, still retrievable, still "present."

- **Authority that still exists may no longer be reachable or current.** A named risk owner may have left the team, changed roles, or lost the delegated authority they held when the asset was authorized — the name field in the record does not know any of that.
- **A decision that was admissible once may not be admissible now.** An approval granted for a customer-facing chatbot with a narrow permission set is not automatically still admissible once that same agent has been silently granted three new tool integrations and a broader data scope.

The practical question this paper investigates is: **does OMG currently re-derive these judgments from current conditions, or does it just check that a record of the original judgment still exists?** These are very different things, and the difference is exactly where runtime governability lives or fails to live.

Section 2 — What OMG Already Has Today

The table below lists only the capabilities directly relevant to runtime governability. Each is classified **IMPLEMENTED** (genuinely computes/re-derives from current state), **PARTIAL** (some real logic, but with a material limitation), or **NOT PRESENT** (no working mechanism exists).

Capability	Purpose	Implementation Evidence	Contribution to Runtime Governability
Governance State Model & Reauthorization Status	Track where an asset sits in its governance lifecycle and whether it is due for renewal	`computeReauthorizationStatus()` (`governanceContinuity.ts`) — pure date-math against `nextReviewDate`, producing Active/Due Soon/Overdue/Expired on every read, never stored	**IMPLEMENTED** for the date dimension of continuity; the single most complete answer to any runtime-governability question in the codebase today
Governance Reassessment Triggers	Name the categories of change that should force re-evaluation (model change, data source change, permission change, control failure, risk threshold breach, etc.)	`ReassessmentTriggerType` (13 values) + `saveReassessmentTrigger()`, which automatically flips an Authorized/Monitoring asset to `Reassessment Required` when a trigger is filed	**PARTIAL** — the taxonomy of "material change" exists and the downstream state flip genuinely works, but nothing in the codebase automatically files a trigger when a change occurs; a human has to know it happened and report it
Evidence Traceability	Determine whether an asset has the evidence its governance state requires	`computeEvidenceReadiness()` — checks evidence presence and expiry	**PARTIAL** — checks that evidence *exists* and *hasn't expired*, but does not check whether it is *sufficient in kind or quantity* for the decision it's meant to support
Decision Reconstruction	Rebuild the full trail of who decided what, when, and why for a given asset	`getDecisionReconstruction()` — assembles the asset's decision history from existing evidence, findings, and outcome records	**IMPLEMENTED** as a retrospective capability — it can show you the history accurately. It does not, by itself, judge whether that history is still admissible today

Capability	Purpose	Implementation Evidence	Contribution to Runtime Governability
Governance Findings	Capture issues discovered about an asset (audit findings, control gaps)	`getFindings()` and the finding-driven inputs to `computeGovernanceOutcome()`	**IMPLEMENTED** as an input signal; findings genuinely drive the outcome ladder (see below)
Corrective Actions	Track remediation work opened against findings or certification gaps	`getCorrectiveActions()` / `getCorrectiveActionsForEntity()`, feeding `computeCertificationReadiness()`	**IMPLEMENTED** as an input signal to certification readiness
Certification Governance	Determine whether an asset's certification is current and sufficient	`computeCertificationReadiness()` — six-factor check including review currency	**PARTIAL** — checks whether the certification's *supporting conditions* (controls mapped, evidence complete, no open findings/CAPAs, review current) hold today, but does not diff the certification against the *original conditions it was granted under* (e.g., was the risk tier the same then as now?)
Agent Governance (Delegation / Tool Grants)	Track what an autonomous agent is permitted to do and through what delegated authority	`delegationScope` (free-text field), `AgentToolGrant` records (`grantedBy`, `createdAt`, no expiry field)	**NOT PRESENT** as a validity mechanism — both are static descriptive records with no parsing, no expiry, and no re-validation logic anywhere in the codebase
Human Oversight	Confirm a human oversight relationship is defined for an asset	`oversightType` field, checked for presence inside `computeGovernanceReadiness()`	**PARTIAL** — confirms an oversight *type* was declared, not that the named human oversight relationship is still active or reachable
Escalation	Surface when a situation warrants escalation to a governance authority	`computeGovernanceOutcome()`'s `'Escalation Recommended'` tier, driven by critical policy violations or critical open findings	**PARTIAL** — the recommendation logic is real and evidenced, but escalation stops at a recommendation; there is no timer, no automatic severity increase with age, and no automatic reassignment
Runtime Monitoring	Continuously watch an asset's operating conditions for governance-relevant change	`governanceState = 'Monitoring'` label; Governance Journey Timeline (chronological aggregation of evidence/findings/CAPAs/certification/incidents)	**PARTIAL** — "Monitoring" today is a static status label plus a retrospective, read-time aggregation of what has already happened. There is no live signal ingestion and no standing background watcher

Capability	Purpose	Implementation Evidence	Contribution to Runtime Governability
Kill Switch	Provide a mechanism to immediately suspend a governed asset	`requestKillSwitch()` / `releaseKillSwitch()`, form-driven, RBAC-gated, synchronously sets `operationalStatus = 'Suspended'`	**IMPLEMENTED** as a human-invoked control. The mechanism itself is solid and audit-logged. It is **NOT PRESENT** as a recommendation-driven action — no computed signal (health score, drift, gates) ever proposes that a kill-switch review is warranted
Composite Governance Signals (Health Score, Gates, Drift)	Summarize an asset's overall governance posture from multiple inputs	`calculateAssetGovernanceHealthScore()` (weighted 5-pillar score), `computeGovernanceGates()` (5-gate PASS/PENDING/FAIL), `detectDrift()` (6-category drift detection with a compound-escalation rule)	**IMPLEMENTED** as advisory signals, each individually well-built and evidenced. **NOT PRESENT**: any composition of these into a single "is continued execution still governable" verdict, and none of them is wired to anything that acts (see Section 4)

Section 3 — Where OMG Already Addresses the Challenge

It is important to state plainly: OMG is not starting from zero on this question. Several areas provide genuinely meaningful coverage, and the architecture underneath them is exactly the right shape to extend.

Governance Continuity is the strongest existing answer. `computeReauthorizationStatus()` does precisely what runtime governability requires in miniature: it takes current date, compares it against a governance-relevant threshold, and produces a live, re-derived status — never a stored flag that can silently go stale. If nothing else in OMG changed, this function alone proves the pattern is achievable and already trusted in production.

Reassessment goes further than a first glance suggests. It is not merely a status field — filing a trigger genuinely changes a live asset's `governanceState` to `Reassessment Required`, which is one of only two truly automatic write-paths found anywhere in the codebase (the other being the kill-switch's own suspension side-effect). The taxonomy of *what counts as a material change* — model change, data source change, permission change, control failure, risk threshold breach, and eight others — is already a complete, sensible list; it does not need to be reinvented.

Findings and Corrective Actions are already load-bearing, not decorative. They are real inputs into `computeGovernanceOutcome()` and `computeCertificationReadiness()`, meaning the escalation ladder and certification status genuinely move when findings are opened or CAPAs remain outstanding. This is a working evidentiary chain, not a checkbox.

Certification Reviews go beyond a one-time stamp: `computeCertificationReadiness()`'s inclusion of review currency means a certification can be knocked out of "Ready" status purely because a scheduled review has lapsed, without anyone touching the certification record itself.

Human Oversight is present as a structural requirement — an asset cannot reach full governance readiness without an oversight type being declared — even though (as Section 4 notes) it does not yet verify that the named oversight relationship is still live.

Kill Switch Controls are a genuinely well-built mechanism: human-initiated, RBAC-gated, audit-logged, and consistent end-to-end (engaging it suspends the asset; releasing it reactivates the asset; nothing else in the codebase can trigger it). This is exactly the "instrument of last resort" a runtime governability model needs to exist — it is present and sound; what is missing (Section 4) is a way to *recommend* its use.

Honest limit: in every one of these areas, the mechanism answers "did the required thing happen, and is it still on file" — not "given everything that has changed, does the thing that happened still mean what it used to mean." That distinction is the entire subject of Section 4.

Section 4 — The Actual Gap

Restating Szilvia's challenge as a checklist of standing questions, and classifying OMG's current ability to answer each automatically:

Already Covered - *Is a reauthorization date approaching or passed?* — Yes, computed live and correctly (`computeReauthorizationStatus()`). - *Has a material-change category been reported for this asset?* — Yes, if a human files it; the downstream consequence (forcing reassessment) is automatic once filed.

Partially Covered - *Is evidence still sufficient?* — OMG checks evidence *exists* and *hasn't expired*. It does not check whether the evidence on file actually matches what the current risk tier, asset type, or certification requires, or whether one stale-but-unexpired record is being asked to justify a materially different asset than the one it was written about. - *Is certification still valid under current conditions?* — OMG re-checks the *supporting checklist* (controls, evidence, findings, CAPAs, review currency) but never diffs the *conditions at certification time* against *conditions now*. - *Is escalation still valid / necessary?* — The recommendation logic is real, but "still valid" implies a re-check over time; today's escalation recommendation is a snapshot at the moment it is computed, with no persistence of whether the underlying condition has since worsened, resolved, or gone stale itself.

Not Covered - *Is authority still reachable?* — Authority fields are name strings whose presence is counted, never whose currency is verified. Nothing checks that the named owner still holds that role. - *Is intervention (reassessment, review, escalation) still admissible given who is authorized to request it?* — No mechanism cross-checks "who is asking for this action" against "who currently holds the authority to request it." - *Is a kill-switch activation justified right now?* — The mechanism to *act* is solid; the mechanism to *recommend* is absent. No computed signal (health score, drift, gates, outcome ladder) ever proposes that a kill-switch review is warranted. - *Is continued execution still permissible, taken as a whole?* — This is the composite question, and nothing composes the individual signals (continuity status, evidence readiness, authority state, certification validity, drift, health score) into a single, live, re-derivable answer. Each signal exists in isolation; none of them talk to each other.

This is the shape of the gap: **OMG has built excellent point-in-time and point-in-existence checks, and one genuinely live continuity check (reauthorization-by-date). It has not yet built a mechanism that treats "has anything changed since the last time this was judged" as a first-class, continuously re-askable question across all the dimensions that matter — evidence, authority, certification, and the composite of all of them together.**

Section 5 — Proposed OMG Evolution

The proposal here is deliberately narrow: a small **Runtime Governability Engine**, built as a pure composition layer over data OMG already has, following the exact architectural conventions already proven in `governanceContinuity.ts`, `readinessFoundation.ts`, and `governanceGatesEngine.ts` — computed on read, never persisted as a new source of truth, advisory only, human-accountable.

Inputs (all already exist in OMG today — no new data collection required): - Asset state at the time of last governance decision (a snapshot already reconstructable via Decision Reconstruction) - Current asset state (model version, data sources, tool grants, risk classification, owner assignments) - Evidence registry contents and ages - Open findings and corrective actions - Certification records and their supporting checklist - Existing composite signals: health score, gates, drift status

Signals the engine would compute (each a small, testable, pure function, in the spirit of `computeReauthorizationStatus`): - *Context Drift Signal* — has anything in the current state materially diverged from the state at last decision? (Extends the existing `ReassessmentTriggerType` taxonomy into an automatic diff, rather than relying solely on a human filing a trigger.) - *Evidence Sufficiency Signal* — does the evidence on file meet a minimum bar (count, type-match, recency) appropriate to the asset's current classification, not just "greater than zero"? - *Authority Currency Signal* — is each named authority role still plausible given what OMG knows about current ownership records, rather than simply "is the field non-empty"? - *Admissibility Signal* — do the conditions that produced the last approval/certification still hold, expressed as a diff rather than a re-run of the original checklist?

Assessments: each signal above resolves independently to one of the same three states used everywhere else in OMG (PASS / PENDING-REVIEW / FAIL-style, matching the existing Gate vocabulary) — deliberately reusing vocabulary practitioners already know from `computeGovernanceGates()` rather than introducing new terminology.

Decision Logic: composition follows the same pattern already used in `computeGovernanceOutcome()`'s outcome ladder — a small ordered set of rules (e.g., "any FAIL on Authority or Evidence escalates the composite status; two or more PENDING signals compound like the existing Drift engine's Compound Drift Rule") rather than a new scoring formula invented from scratch.

Governability Status: a single, human-readable composite state per asset — *Governable*, *Governable with Conditions*, *Review Required*, *Not Currently Governable* — computed live, displayed wherever the existing health score and gates are already shown (asset detail page, Journey Explorer), and explicitly **never gating anything**. Consistent with every existing engine in OMG, it recommends; it does not block, disable, or auto-suspend.

Auditability: because the engine is a pure function over existing, already-audited data, every Governability Status carries an explainable `reasons[]` array in the same style as `computeGovernanceOutcome()` — a reviewer can always see exactly which signal(s) drove the status, and Decision Reconstruction already provides the historical trail needed to explain *why* the prior decision was made in the first place.

What this proposal deliberately avoids: no new persisted table, no new domain, no new workflow, no automatic enforcement action, no industry-specific control logic. It is a composition of signals OMG already computes, expressed through vocabulary OMG already uses.

Section 6 — Common Product Baseline

The following should become reusable OMG baseline capability, because each is industry-agnostic, reusable across any customer's asset population, and expressed purely in terms of governance mechanics rather than any specific regulatory regime:

- **Context Drift, Evidence Sufficiency, Authority Currency, and Admissibility signals** as described in Section 5 — these are generic governance-math functions, no different in kind from the health score or gates engines already shipped to every customer.
- **The Governability Status composite** and its explainable reasons — a generalized "is this still good" summary is exactly as universal as the existing readiness/health/gates summaries.
- **The extended context-diff mechanism** (comparing decision-time state to current state) — this is pure computation over `AIAsset`, `EvidenceRecord`, and `CertificationRecord` fields that already exist identically for every

customer.

- **The kill-switch-recommendation link** (surfacing, not triggering, a kill-switch review when the composite status warrants it) — the underlying kill-switch mechanism is already common baseline; recommending its use from a common signal keeps it common.

These belong in the core product because fragmenting them into per-customer builds would recreate, N times, logic that has nothing customer-specific in it — the same argument that already justifies keeping the health score and gates engines in core today.

Section 7 — Customer Configuration

The following should remain configurable per customer, because the *mechanism* is universal but the *threshold or policy value* legitimately varies by industry, risk appetite, and regulatory environment:

- **Evidence sufficiency thresholds** — minimum evidence count and required evidence types per asset classification or risk tier (a regulated-industry customer needs a stricter bar than an internal-tooling customer).
- **Reauthorization and review windows** — the existing 30/90-day thresholds in `computeReauthorizationStatus()` are already a natural candidate for a configurable table, following the exact pattern already used for `REVIEW_FREQUENCY_DAYS`.
- **Which authority fields are mandatory** — not every customer's org structure has a distinct compliance owner; the mandatory-fields list should be configurable per tenant rather than a fixed array.
- **Escalation severity mappings and "material change" definitions** — which conditions count as critical enough to trigger escalation, and which change categories matter most, should be tunable per customer governance policy, using the same `isEnabled()`/playbook pattern already present in the codebase.
- **Governability Status thresholds** — how many PENDING signals compound into a Review Required vs. a Not Currently Governable verdict is a policy choice, not a universal constant.

Section 8 — Customer-Specific Implementation

The following should remain entirely customer-owned, outside OMG's core product surface:

- **Organization-specific approval chains** — the actual sequence of named roles/people who must sign off, which varies by company structure and cannot be generalized.
- **Regulatory mappings** — how a given customer's specific regulatory obligations map onto OMG's generic control/evidence framework is inherently customer- and jurisdiction-specific.
- **Intervention policies** — the customer's own operational runbook for what happens once OMG recommends "Review Required" or "Not Currently Governable" (who is paged, what system is used, what the remediation SLA is) belongs to the customer's operations, not to OMG's product.
- **Domain-specific controls** — industry-particular control requirements (e.g., specific to healthcare, financial services, or defense) should be expressed as customer-authored entries within OMG's existing generic control/evidence structures, not as bespoke OMG code.
- **Runtime event connectors** (CloudWatch, Datadog, ServiceNow, Jira, or any other monitoring/ITSM tool) — no such integrations exist in OMG today, and building them bespoke per customer would turn OMG into a custom-services platform. If pursued at all, OMG should offer a generic, documented event-ingestion contract

and let each customer wire their own tooling into it.

Section 9 — Answer to Szilvia's Question

Question	Current State	Gap	Recommended Evolution
Evidence admissibility — is evidence on file still sufficient?	Checks presence and expiry only (`assetEvidence.length > 0`)	No minimum count, no type-matching, no relevance-to-current-classification check	Evidence Sufficiency Signal (Section 5) — configurable minimum bar per classification
Authority admissibility — is the named authority still valid?	Counts non-empty name fields; never verifies currency	No mechanism confirms a named owner still holds that role or authority	Authority Currency Signal (Section 5), reusing existing ownership data
Escalation admissibility — is an escalation recommendation still warranted?	Computed live from critical violations/findings at the moment of calculation	No persistence of whether the underlying condition has changed since first flagged; no timer or staleness check on the recommendation itself	Fold into the Governability Status composite; treat an unresolved escalation as a Context Drift input
Intervention admissibility — can this action legitimately be requested right now, by whoever is requesting it?	No cross-check exists between "who is requesting an action" and "who currently holds authority to request it"	Authority Currency Signal is a prerequisite this doesn't yet have	Authority Currency Signal, applied at the point an intervention is proposed, not only at asset-level display
Kill-switch admissibility — is activation justified right now?	Kill-switch mechanism itself is sound and human-gated; nothing recommends its use	No computed signal ever proposes a kill-switch review	Governability Status "Not Currently Governable" verdict surfaces a kill-switch review recommendation — never an automatic activation
Continued execution admissibility — should this asset keep running, taken as a whole?	No composite exists; individual signals (health, gates, drift, continuity) are computed independently and never combined	This is the central missing piece — a composition gap, not a data gap	The Governability Status composite (Section 5) — the single highest-leverage addition proposed in this paper

Direct answer: OMG can today answer one of these six questions well (reauthorization timing, which underlies part of "continued execution admissibility"), partially answer three (evidence, certification/escalation, and the authority-adjacent oversight check), and cannot yet answer two (authority currency and the composite continued-execution verdict). Every gap identified traces to a **missing composition layer**, not to missing underlying data — which means the proposed evolution in Section 5 is an extension of existing OMG architecture, not a new governance domain.

Section 10 — Practitioner Review Questions

1. Does the proposed *Governability Status* (Governable / Governable with Conditions / Review Required / Not Currently Governable) map cleanly onto how your organization already talks about runtime risk, or would different labels reduce ambiguity for your governance committees?

2. Is a four-signal composite (Context Drift, Evidence Sufficiency, Authority Currency, Admissibility) the right granularity, or are there additional signal categories your practitioners would consider indispensable before trusting this status?

3. For the Context Drift Signal: is comparing "state at last decision" to "state now" sufficient, or does meaningful drift detection require intermediate checkpoints rather than a single two-point diff?

4. For Evidence Sufficiency: what would your organization consider a defensible minimum evidence bar per risk tier, and should that bar itself require governance sign-off before being configured?

5. For Authority Currency: what is an acceptable way to verify a named authority is "still reachable" without OMG needing a live HR/identity integration — is a periodic manual attestation an adequate interim answer?

6. Should a "Not Currently Governable" verdict be visible only to governance roles, or should it also be visible to the asset's operational owner, given OMG's human-accountable (no auto-block) philosophy?

7. Is it acceptable that this entire engine remains advisory-only — i.e., that even a "Not Currently Governable" status never itself suspends or blocks the asset — or does your governance model require a stronger consequence at some threshold?

8. How should the compounding logic behave when signals disagree — e.g., strong Evidence Sufficiency but failing Authority Currency? Should any single FAIL dominate, or should the composite reflect a weighted judgment?

9. Does the proposed customer-configuration boundary (thresholds and policy values configurable; mechanism itself common) match where your organization would want to exercise control, or are there mechanism-level behaviors you would also need to customize?

10. What blind spots, if any, do you see in restricting this paper's scope to Continuity, Reassessment, Evidence, Decision Reconstruction, Findings, Corrective Actions, Certification, Agent/Tool Governance, Human Oversight, Escalation, Monitoring, and Kill Switch — is there a runtime-governability-relevant capability outside this list that should have been included?

Appendix — Classification Summary

#	Item	Status
1	What OMG already implements	Reauthorization status (date-based continuity); reassessment trigger taxonomy and its automatic state flip; findings and corrective actions as live inputs to certification/outcome computation; decision reconstruction; kill-switch mechanism (human-invoked)

#	Item	Status
2	What OMG partially implements	Evidence readiness (presence/expiry only); certification readiness (checklist only, no decision-time diff); escalation recommendation (real but not persisted/timed); human oversight (declared but not verified live); runtime monitoring (static label + retrospective timeline, no live watcher)
3	What OMG does not yet implement	Authority currency re-derivation; intervention/escalation admissibility cross-checked against current authority; kill-switch activation recommendation; a composite continued-execution (Governability Status) verdict
4	What we propose to add	A Runtime Governability Engine composing four new signals (Context Drift, Evidence Sufficiency, Authority Currency, Admissibility) into one live, explainable, advisory Governability Status
5	What remains customer configurable	Evidence sufficiency thresholds; reauthorization/review windows; mandatory authority fields; escalation severity and material-change definitions; Governability Status compounding thresholds
6	What remains customer specific	Approval chains; regulatory mappings; intervention runbooks/SLAs; domain-specific controls; any bespoke runtime event connectors

This paper is confined to the Runtime Governability question raised by Szilvia Földesné Ambrus. It does not describe, and should not be read as describing, the full OMG capability set, release history, or product roadmap.